Package 'miMeta'

October 21, 2026

Type Package

Title Meta-analysis of microbiome association studies

Version 2.1.0

Description
This R package implements new methods for meta-analysis of microbiome association studies that respect the unique features of microbiome data such as compositionality. Melody combines summary statistics from multiple microbiome association studies to identify microbial signatures whose associations generalize across studies. SMESH groups contexts (for example, diseases or tumor types) into latent clusters with similar association profiles, and identifies microbial signatures that are shared by all clusters or specific to a subset of clusters.

Date 2026-10-19

Reference Wei Z, Chen G, Tang ZZ. Melody: meta-analysis of microbiome association studies for discovering generalizable microbial signatures. Genome Biology. 2025;26:245. doi:10.1186/s13059-025-03721-4.
Wei Z, Chen G, Tang ZZ. Structure-learning meta-analysis resolves cross-context heterogeneity in microbiome associations. Submitted.

License GPL (>= 3) | file LICENSE

Imports MASS,
Matrix,
brglm2,
abess,
UpSetR,
dplyr,
ggplot2,
doParallel,
parallel,
foreach,
PALM (>= 0.1.1)

RoxygenNote 7.3.3

Suggests knitr,
rmarkdown,
BiocStyle,
tidyverse

Remotes PALM=github::ZjpWei/PALM_package

BugReports https://github.com/ZjpWei/miMeta/issues

URL https://github.com/ZjpWei/miMeta

VignetteBuilder knitr

Encoding UTF-8

LazyData true

Depends R (>= 4.3.0)

Contents

CRC_data . . . 2
melody . . . 3
melody.get.summary . . . 6
melody.meta.summary . . . 8
melody.null.model . . . 9
plot_signatures . . . 10
signature_table . . . 11
sim_data . . . 13
smesh . . . 14
smesh.summary . . . 18

`CRC_data` *Data from two metagenomic studies of colorectal cancer (CRC)*

Description

The "CRC_data" includes "CRC_abd" and "CRC_meta". "CRC_abd" is a sample-by-feature matrix of sequencing read counts from the two studies including 267 species under order "Clostridiales". The "CRC_meta" is a data frame including the sample-level variables from the two studies.

Usage

```
data(CRC_data)
```

Format

An object of class `list` of length 2.

Source

<https://github.com/zellerlab/crc_meta>

References

Wirbel, Jakob et al. Nat Med. 2019 Apr;25(4):679-689.

Examples

```
library("miMeta")
data("CRC_data")
CRC_abd <- CRC_data$CRC_abd
CRC_meta <- CRC_data$CRC_meta
```

melody *Meta-analysis for selecting microbial signatures associated with covariates of interest*

Description

Melody is a meta-analysis method designed to account for the unique features of compositional microbiome data in selecting microbial signatures.

Usage

```
melody(
  feature.counts,
  covariate.interest,
  covariate.adjust = NULL,
  cluster = NULL,
  depth.filter = 0,
  prev.filter = 0.1,
  ref = NULL,
  tune.path = c("gsection", "sequence"),
  tune.size.sequence = NULL,
  tune.size.range = NULL,
  tune.type = c("BIC", "HBIC", "KBIC", "EBIC"),
  tol = 0.001,
  NMAX = 20,
  parallel.core = NULL,
  output.best.one = TRUE,
  verbose = FALSE
)
```

Arguments

`feature.counts` A named list of sample-by-feature matrices of nonnegative sequencing read counts, with one matrix per study. The list names must be the study identifiers; matrix row names and column names must be sample and microbial-feature identifiers, respectively. Row sums are used as sequencing-depth measures. The counts should not be converted to proportions or normalized. Missing values are not allowed.

`covariate.interest` A list of matrices for single or multiple covariates of interest. Each element of the list pertains to one study, and the name of each element must be the study ID. In a matrix of a given study, each row is a sample, and each column is a covariate of interest and can only be a numeric vector. The set of covariates can be different among studies. In a given study, the order of samples should be matched with the order in "feature.counts". No missing value is allowed.

`covariate.adjust` A list of data frames for covariates that will be adjusted for in the model. Each element of the list pertains to one study, and the name of each element must be the study ID. In a data frame of a given study, each row is a sample, and each column is a covariate to be adjusted and can only be a factor or numeric vector. The set of covariates can be different among studies. In a given study, the order

of samples should be matched with the order in "feature.counts". No missing value is allowed. Default is NULL (not adjusting for any covariates)

`cluster` A list of vectors of variable that define the sample cluster for studies with correlated samples. Each element of the list pertains to one study with correlated samples, and the name of each element must be the study ID. The order of samples should be matched with the order in "feature.counts" in the corresponding study. For example, the values of this variable are subject IDs if each subject has multiple correlated samples (e.g., measured in a longitudinal study). Default is NULL (all samples in all studies are independent, so no need to provide this input.).

`depth.filter` A cutoff value to remove samples with sequencing depth less than or equal to the cutoff. Default is 0.

`prev.filter` A cutoff value to remove microbial features with prevalence (proportion of nonzero observations) less than or equal to the cutoff. This cutoff is applied to each study. So, a feature could be removed in a subset of the studies. The cutoff value must be in the range of 0-1. Default is 0.1.

`ref` A feature ID or a vector of feature IDs. If a feature ID is provided, this microbial feature will be used as the reference in all studies when generating summary statistics. If a vector of feature IDs is provided, each feature in the vector will be the reference for a study. Therefore, the length of the vector should be equal to the number of studies and have study IDs as names. Default is NULL (the function will automatically pick reference for each study).

`tune.path` Method to choose the optimal subset size in the best subset selection. If tune.path is "sequence", we solve the best subset selection problem for each size in "tune size sequence". If tune.path is "gsection", we solve the best subset selection problem with the size ranged in tune.size.range, where the considered size sequence within the range is determined by golden section search. Default is "gsection".

`tune.size.sequence`
An integer vector containing the subset sizes to be considered. Only used when tune.path is "sequence". Default is 0:round(K/2), where K is number of microbial features.

`tune.size.range`
An integer vector with two elements that define the range of the size. Default is c(0, K/2)

`tune.type` Type of information criterion for choosing the best subset size. Available options are "BIC", "HBIC", "KBIC" and "EBIC". Default is "BIC".

`tol` Convergence tolerance in the search for the best delta tuning parameters. Default is 1e-3.

`NMAX` The maximum number of iterations in the search for the best delta tuning parameters. Default is 20.

`parallel.core` The number of cores to be concurrently used for generating summary statistics. It's an integer between 1 and cl - 1 (cl is the total core number). Default is cl - 1.

`output.best.one`
Whether only output the best model. Default is TRUE.

`verbose` Whether to print verbose information. Default is FALSE. (see details in Value)

Details

Melody first generates summary statistics (microbiome association coefficient estimates and their variances) for individual studies. Melody then combines the summary statistics across studies to select microbial signatures. In particular, the selection of signature is operated through a best-subset selection. There are two sets of tuning parameters in the best subset selection including the subset size (i.e. the number of microbial features selected) and delta's which are introduced to recover the absolute-abundance coefficients from the relative-abundance coefficients. (details in reference Wei et al.)

Value

Returns a list with each component for a covariate of interest.

If output.best.one=TRUE (default), the component includes the following elements to deliver the meta-analysis results under the best model over the subset sizes considered.

`coef`	A coefficient vector that contains the meta association coefficient estimate (at the absolute-abundance level) under the best subset size. The vector names are microbial features IDs. The microbial features with nonzero coefficients are the selected signatures.
`delta`	A vector that contains the best values of the delta tuning parameters under the best subset size. The vector names are in the format "<study ID>_<reference microbial feature ID>"
`dev`	The value of the deviance for the best subset size.
`ic`	The value of the information criterion (the type of information criterion is specified in tune.type) for the best subset size.

If output.best.one=FALSE, the component includes the following elements to deliver the meta-analysis results under all models over the subset sizes considered. (depends on tune.path, see details in Arguments)

`coef`	A coefficient matrix with each column contains the meta association coefficient estimate (at the absolute-abundance level) under a subset size. The row names are microbial features IDs and the column names are the subset sizes considered.
`delta`	A matrix with each column contains the best values of the delta tuning parameters under a subset size. The row names are in the format "<study ID>_<reference microbial feature ID>" and the column names are the subset sizes considered.
`dev`	A vector contains the values of the deviance for the subset sizes considered.
`ic`	A vector contains the values of the information criterion (the type of information criterion is specified in tune.type) for the subset sizes considered.

If verbose=TRUE, print out information about the progress of the meta-analysis. In addition, it will generate an intersection plot showing the number of microbial features shared among studies and scatter plots for the meta-coefficient estimates for the top 5 covariates of interest associated with most microbial features.

Author(s)

Zhoujingpeng Wei, Guanhua Chen, Zheng-Zheng Tang

References

Wei Z, Chen G, Tang ZZ. Melody: meta-analysis of microbiome association studies for discovering generalizable microbial signatures. Genome Biology. 2025;26:245. doi:10.1186/s13059-025-03721-4.

See Also

melody.get.summary, melody.meta.summary, melody.null.model

Examples

```
library("miMeta")
data("CRC_data")
CRC_abd <- CRC_data$CRC_abd
CRC_meta <- CRC_data$CRC_meta

########## Generate summary statistics ##########
feature.counts <- list()
covariate.interest <- list()
for(d in unique(CRC_meta$Study)){
  feature.counts[[d]] <- CRC_abd[CRC_meta$Sample_ID[CRC_meta$Study == d],]
  disease <- as.numeric(CRC_meta$Group[CRC_meta$Study == d] == "CRC")
  names(disease) <- CRC_meta$Sample_ID[CRC_meta$Study == d]
 covariate.interest[[d]] <- matrix(disease, ncol = 1, dimnames = list(names(disease), "disease"))
}

meta.result <- melody(feature.counts = feature.counts, covariate.interest = covariate.interest,
                      ref = "Coprococcus catus [ref_mOTU_v2_4874]")
```

---

melody.get.summary *Generate Melody summary statistics for individual studies*

Description

This function directly takes the output object from the "melody.null.model" function and constructs summary statistics for covariates of interest in each study. The output can be directly used by the function "melody.meta.summary" to perform meta-analysis.

Usage

```
melody.get.summary(
  null.obj,
  covariate.interest,
  cluster = NULL,
  parallel.core = NULL,
  verbose = FALSE
)
```

Arguments

null.obj	The output of function "melody.null.model".
...	See function melody. In covariate.interest and cluster, the order of samples should be matched with the order in feature.counts used in the "melody.null.model" function.

Value

Returns a list with each component for a study. The component includes the following elements.

ref	Reference feature ID for the study.
est	A matrix contains the relative-abundance association coefficient estimates for the study. The row names are microbial feature IDs and the column names are the covariates of interest IDs.
var	A matrix contains the variances of the relative-abundance association coefficient estimates for the study. The row names are microbial feature IDs and the column names are the covariates of interest IDs.
n	Sample size for the study.

See Also

melody.null.model, melody.meta.summary, melody

Examples

```
library("miMeta")
data("CRC_data")
CRC_abd <- CRC_data$CRC_abd
CRC_meta <- CRC_data$CRC_meta

########## Generate summary statistics ##########
feature.counts <- list()
covariate.interest <- list()
for(d in unique(CRC_meta$Study)){
  feature.counts[[d]] <- CRC_abd[CRC_meta$Sample_ID[CRC_meta$Study == d],]
  disease <- as.numeric(CRC_meta$Group[CRC_meta$Study == d] == "CRC")
  names(disease) <- CRC_meta$Sample_ID[CRC_meta$Study == d]
 covariate.interest[[d]] <- matrix(disease, ncol = 1, dimnames = list(names(disease), "disease"))
}

 null.obj <- melody.null.model(feature.counts = feature.counts, ref = "Coprococcus catus [ref_mOTU_v2_4874]")

 summary.stats <- melody.get.summary(null.obj = null.obj, covariate.interest = covariate.interest)
```

`melody.meta.summary` *Meta-analyze summary statistics across studies*

Description

This function directly takes the summary statistics output from the "melody.get.summary" function and combines the summary statistics across studies for selecting microbial signatures associated with each covariate of interest.

Usage

```
melody.meta.summary(
  summary.stats,
  tune.path = c("gsection", "sequence"),
  tune.size.sequence = NULL,
  tune.size.range = NULL,
  tune.type = c("BIC", "HBIC", "KBIC", "EBIC"),
  output.best.one = TRUE,
  tol = 0.001,
  NMAX = 20,
  verbose = FALSE
)
```

Arguments

`summary.stats`	The output of function "melody.get.summary".
`...`	See function melody.

Value

Same output as the function "melody".

See Also

`melody.null.model`, `melody.get.summary`, `melody`

Examples

```
library("miMeta")
data("CRC_data")
CRC_abd <- CRC_data$CRC_abd
CRC_meta <- CRC_data$CRC_meta

########## Generate summary statistics ##########
feature.counts <- list()
covariate.interest <- list()
for(d in unique(CRC_meta$Study)){
  feature.counts[[d]] <- CRC_abd[CRC_meta$Sample_ID[CRC_meta$Study == d],]
  disease <- as.numeric(CRC_meta$Group[CRC_meta$Study == d] == "CRC")
  names(disease) <- CRC_meta$Sample_ID[CRC_meta$Study == d]
 covariate.interest[[d]] <-matrix(disease, ncol = 1, dimnames = list(names(disease), "disease"))
}
```

```
null.obj <- melody.null.model(feature.counts = feature.counts, ref = "Coprococcus catus [ref_mOTU_v2_4874]")

summary.stats <- melody.get.summary(null.obj = null.obj, covariate.interest = covariate.interest)

########## Meta-analysis ##########
meta.result <- melody.meta.summary(summary.stats = summary.stats)
```

`melody.null.model` *Get information from the null model*

Description

The "melody.null.model" function computes the estimated mean microbial feature proportions and residuals of the relative abundance under the null model of no associations for each study (so this function does not need the covariate.interest information). The output of this function will be fed into the "melody.get.summary" function to construct summary statistics for covariates of interest in each study.

Usage

```
melody.null.model(
  feature.counts,
  covariate.adjust = NULL,
  depth.filter = 0,
  prev.filter = 0.1,
  ref = NULL,
  parallel.core = NULL
)
```

Arguments

`feature.counts`	A named list of sample-by-feature matrices of nonnegative sequencing read counts, with one matrix per study. The list names must be the study identifiers; matrix row names and column names must be sample and microbial-feature identifiers, respectively. Row sums are used as sequencing-depth measures. The counts should not be converted to proportions or normalized. Missing values are not allowed.
`...`	See function melody.

Value

Returns a list with each component for a study. The component includes the following elements.

`ref`	Reference feature ID for the study.
`p`	A matrix of the estimated mean microbial feature proportions for the study.
`res`	A matrix of the residuals of the relative abundance for the study.
`N`	A vector of microbiome sequencing depth for the study.
`X`	A data frame of the cleaned covariate.adjust for the study.
`rm.sample.idx`	The index of the removed samples for the study.

See Also

melody.get.summary, melody.meta.summary, melody

Examples

```
library("miMeta")
data("CRC_data")
CRC_abd <- CRC_data$CRC_abd
CRC_meta <- CRC_data$CRC_meta

########## Generate summary statistics ##########
feature.counts <- list()
for(d in unique(CRC_meta$Study)){
  feature.counts[[d]] <- CRC_abd[CRC_meta$Sample_ID[CRC_meta$Study == d],]
}

null.obj <- melody.null.model(feature.counts = feature.counts, ref = "Coprococcus catus [ref_mOTU_v2_4874]")
```

---

plot_signatures *Heat map of the signatures selected by SMESH*

Description

Draws the final cluster-level effect of every selected signature, one row per signature and one column per cluster, grouped by sharing category.

Usage

```
plot_signatures(
  fit,
  covariate = NULL,
  title = "Cluster-level association effects"
)
```

Arguments

fit	A fitted object returned by smesh or smesh.summary: either the full list, with one element per covariate of interest, or one of its elements.
covariate	The covariate of interest to plot when fit holds several. Default is NULL, which plots the first.
title	The plot title. Default is "Cluster-level association effects".

Value

A ggplot object. Rows follow the order of signature_table. Cells where the feature is estimable but not selected (effect zero) are drawn in the neutral colour of the scale; cells where no context assigned to the cluster could estimate the feature (NA) are grey.

Author(s)

Zhoujingpeng Wei, Guanhua Chen, Zheng-Zheng Tang

See Also

signature_table, smesh, smesh.summary

Examples

```
library("miMeta")
data("sim_data")
## 10 simulated contexts in 2 latent clusters, 100 features, 30 true signatures
feature.counts <- sim_data$sim_abd
sim_meta <- sim_data$sim_meta

## Disease status of each sample, one single-column matrix per context
covariate.interest <- lapply(names(feature.counts), function(d){
  m <- sim_meta[sim_meta$Context == d, ]
  matrix(m$Group, ncol = 1, dimnames = list(m$Sample_ID, "disease"))
})
names(covariate.interest) <- names(feature.counts)

## Each context is one study, so `context` is left at NULL. G = 2, the true
## number of clusters, keeps the example quick; with the default G = NULL,
## G is selected by the minimum GIC.
fit <- smesh(feature.counts = feature.counts,
             covariate.interest = covariate.interest, G = 2)
plot_signatures(fit)
```

---

signature_table *Signature table of a fitted SMESH model*

Description

Summarizes the microbial signatures selected by SMESH, one row per feature: its sharing category, the clusters in which it is selected, its final effect in each cluster, and whether each cluster could estimate it.

Usage

```
signature_table(fit, covariate = NULL, selected.only = TRUE)
```

Arguments

fit	A fitted object returned by smesh or smesh.summary: either the full list, with one element per covariate of interest, or one of its elements.
covariate	The covariate of interest to summarize when fit holds several. Default is NULL, which uses the first.
selected.only	Whether to list only the selected signatures, that is, features with a nonzero effect in at least one cluster. Default is TRUE; set it to FALSE to list every feature in the fit.

Details

The sharing category is decided by the clusters in which the effect is nonzero, among the clusters in which the feature is estimable. A signature is all-cluster shared when it is selected in every cluster in which it is estimable and at least two clusters can estimate it (with a single cluster, every selected feature is all-cluster shared); its effect magnitudes and directions may differ across clusters. A cluster in which no assigned context could estimate the feature has an NA effect and is not counted; the `estimable_` columns show when that is the case, and `active_clusters` then lists only the clusters that selected the feature. A signature selected in exactly one cluster is cluster-specific.

Value

A data frame with one row per feature, in the order drawn by `plot_signatures`: all-cluster-shared signatures first, by the sum of the cluster effects; then subset-shared and then cluster-specific signatures, grouped by the pattern of active clusters and, within one pattern, by the sum of the cluster effects. The columns are:

`feature` The microbial feature identifier.

`sharing` "all-cluster shared" (selected in every cluster in which it is estimable), "subset-shared" (selected in more than one cluster but not in every cluster in which it is estimable), or "cluster-specific" (selected in exactly one cluster). With `selected.only = FALSE`, unselected features are "not selected".

`active_clusters` The clusters in which the feature is selected, separated by "|", for example "C1|C3".

`effect_C1, effect_C2, ...` The final effect estimate in each cluster, as in the `effects` component of the fit: nonzero where the feature is selected, zero where it is estimable but not selected, and NA where no context assigned to the cluster provided an estimable association summary for it.

`estimable_C1, estimable_C2, ...` Whether at least one context assigned to the cluster provided an estimable association summary for the feature.

Author(s)

Zhoujingpeng Wei, Guanhua Chen, Zheng-Zheng Tang

See Also

`plot_signatures`, `smesh`, `smesh.summary`

Examples

```
library("miMeta")
data("sim_data")
## 10 simulated contexts in 2 latent clusters, 100 features, 30 true signatures
feature.counts <- sim_data$sim_abd
sim_meta <- sim_data$sim_meta

## Disease status of each sample, one single-column matrix per context
covariate.interest <- lapply(names(feature.counts), function(d){
  m <- sim_meta[sim_meta$Context == d, ]
  matrix(m$Group, ncol = 1, dimnames = list(m$Sample_ID, "disease"))
})
names(covariate.interest) <- names(feature.counts)
```

```
## Each context is one study, so `context` is left at NULL. G = 2, the true
## number of clusters, keeps the example quick; with the default G = NULL,
## G is selected by the minimum GIC.
fit <- smesh(feature.counts = feature.counts,
             covariate.interest = covariate.interest, G = 2)
sig <- signature_table(fit)
head(sig)
table(sig$sharing)
```

`sim_data` *Simulated data of 10 contexts in two latent clusters*

Description

The "sim_data" is a small simulated data set for the examples of the SMESH functions. It includes "sim_abd", "sim_meta" and "sim_truth". The 10 contexts fall into two latent clusters of five contexts each, and 30 of the 100 microbial features are true signatures: 15 are all-cluster shared, 8 are specific to cluster C1 and 7 are specific to cluster C2.

Usage

```
data(sim_data)
```

Format

An object of class `list` of length 3.

Details

- "sim_abd" is a list of sample-by-feature matrices of sequencing read counts, one per context, each with 100 samples (50 controls and 50 cases) and 100 features.
- "sim_meta" is a data frame of the sample-level variables: "Sample_ID", matching the row names of "sim_abd"; "Context", matching the names of "sim_abd"; and "Group", the disease status, 1 for a case and 0 for a control.
- "sim_truth" is the truth the data were simulated from: "cluster", the latent cluster of each context, and "signatures", a data frame of the 30 true signatures with their sharing category, fold change and direction.

The counts were simulated with MIDASim from 10 colorectal neoplasia metagenomic studies, each context inheriting the distributional and dependence structure of one study, following the simulation design of the SMESH paper. Association signals were introduced by multiplying the mean proportions of the true signatures in one group by a fold change, drawn from Uniform(2, 2.5) for all-cluster-shared signatures and Uniform(5, 7) for cluster-specific signatures, with 60 percent of the signatures enriched in cases.

References

He M, Zhao N, Satten GA. MIDASim: a fast and simple simulator for realistic microbiome data. Microbiome. 2024;12:135. doi:10.1186/s40168-024-01822-z.

Wei Z, Chen G, Tang ZZ. Structure-learning meta-analysis resolves cross-context heterogeneity in microbiome associations. Submitted.

Examples

```
library("miMeta")
data("sim_data")
sim_abd <- sim_data$sim_abd
sim_meta <- sim_data$sim_meta
sim_data$sim_truth$cluster
head(sim_data$sim_truth$signatures)
```

`smesh` *Structure-learning meta-analysis for discovering shared and context-dependent microbial signatures*

Description

Runs the end-to-end SMESH pipeline: generates study-level association estimates and standard errors using PALM, combines studies representing the same context, learns latent context clusters, and estimates all-cluster-shared and context-dependent microbial signatures.

Usage

```
smesh(
  feature.counts,
  covariate.interest,
  context = NULL,
  covariate.adjust = NULL,
  cluster = NULL,
  depth.filter = 0,
  prev.filter = 0.1,
  correct = c("tune", "median"),
  min.context = 1,
  G = NULL,
  support.ic = c("BIC", "HBIC", "KBIC", "EBIC"),
  tol = 0.001,
  NMAX = 20,
  nstart = 20,
  verbose = FALSE
)
```

Arguments

`feature.counts` A named list of sample-by-feature matrices of nonnegative sequencing read counts, with one matrix per study. The list names must be the study identifiers; matrix row names and column names must be sample and microbial-feature identifiers, respectively. Row sums are used as sequencing-depth measures. The counts should not be converted to proportions or normalized. Missing values are not allowed.

`covariate.interest`
A named list of matrices for single or multiple covariates of interest, with one element per study named by the study identifier. In a matrix of a given study, each row is a sample and each column is a covariate of interest and can only be

a numeric vector. In a given study, the order of samples should be matched with the order in "feature.counts". No missing value is allowed.

`context` A vector that maps studies to contexts, with study identifiers as names and context labels as values. Studies sharing a context label are combined by fixed-effect meta-analysis into one context-level summary before the clusters are learned, which is how several studies of the same disease or tumor type are combined. Default is NULL, in which case every study is its own context and no combining is needed.

`covariate.adjust` A named list of data frames for covariates that will be adjusted for in the model, with one element per study named by the study identifier. In a data frame of a given study, each row is a sample and each column is a covariate to be adjusted and can only be a factor or numeric vector. In a given study, the order of samples should be matched with the order in "feature.counts". No missing value is allowed. Default is NULL (not adjusting for any covariates).

`cluster` A named list of vectors that define the sample clusters for studies with correlated samples, with one element per study that has correlated samples, named by the study identifier. The order of samples should be matched with the order in "feature.counts" in the corresponding study. For example, the values of this variable are subject IDs if each subject has multiple correlated samples. Default is NULL (all samples in all studies are independent).

`depth.filter` A cutoff value to remove samples with sequencing depth less than or equal to the cutoff. Default is 0.

`prev.filter` A cutoff value to remove microbial features with prevalence (proportion of nonzero observations) less than or equal to the cutoff. This cutoff is applied to each study, so a feature could be removed in a subset of the studies. The cutoff value must be in the range of 0-1. Default is 0.1.

`correct` The bias correction used by PALM when generating the per-study summary statistics. Available options are "tune" and "median". Default is "tune"; "median" is faster and can be used for exploratory runs.

`min.context` The minimum number of contexts that must be able to estimate a feature for that feature to enter the fit. Default is 1, which keeps every feature that at least one context can estimate. Raising it restricts the analysis to features observed more widely.

`G` The number of context clusters. Default is NULL, in which case G is selected by the minimum generalized information criterion (GIC): for each candidate G from 1 to $\min(L-1, \mathrm{round}(2L/3))$, where L is the number of contexts, the sparse association model is first selected with `support.ic`, and the G whose selected model has the minimum GIC is kept, with ties resolved in favor of the smaller G. If a number is supplied, that G is fitted directly; it must be smaller than the number of contexts.

`support.ic` The information criterion used to select the sparse association model, that is, which cluster-level coefficients are nonzero, conditional on a candidate G. Available options are "BIC", "HBIC", "KBIC" and "EBIC". Default is "BIC". Selection of G is performed by the outer GIC procedure described under `G`; the penalty in that GIC is also computed with `support.ic`.

`tol` Convergence tolerance for the mixture fit. Default is 1e-3.

`NMAX` The maximum number of iterations in the mixture fit. Default is 20.

`nstart` The number of random initializations of the context clustering. Default is 20. SMESH uses multiple random starts to reduce sensitivity to local optima; see the Details of `smesh.summary`.

`verbose` Whether to print the progress of the analysis. Default is FALSE.

Details

The function runs three steps. First it fits a PALM null model on the read counts and generates association summary statistics for every study. Second, if "context" is supplied, the studies belonging to one context are combined by a fixed-effect meta-analysis into a single context-level estimate and standard error. Each context therefore contributes one association profile to the clustering, while evidence from multiple studies is reflected through increased precision. Third, the context-level summaries are passed to `smesh.summary`, which learns the clusters and the signatures.

To reuse summary statistics you already have, or to produce them with a different association model, call `smesh.summary` directly.

Value

A named list with one element for each covariate of interest. Each element contains the following components:

`effects` A numeric matrix containing the final cluster-level association effect estimates. Rows correspond to microbial features and columns correspond to the learned context clusters. The estimates are obtained from the post-selection refit. A nonzero entry indicates that the feature was selected as associated in that cluster, whereas zero indicates that it was not selected in a cluster for which the feature was estimable. `NA` indicates that no context assigned to the cluster provided an estimable association summary for that feature. A feature with at least one nonzero entry is a selected microbial signature, and its pattern of nonzero entries determines whether it is all-cluster shared, subset-shared, or cluster-specific. A feature is all-cluster shared when its association is selected as nonzero in every cluster in which it is estimable; its effects need not be equal and may differ in magnitude and direction across clusters. `signature_table` lists the selected signatures with their sharing categories.

`context_assignments` A data frame with one row per context and the columns `context`, `cluster`, and `posterior_probability`. The `cluster` column gives the cluster each context is assigned to in the reported fit, and `posterior_probability` gives the posterior membership probability of that cluster under the fitted mixture model. Cluster labels are consistent across all components of the fitted object but are otherwise arbitrary up to permutation.

`posterior_membership` A numeric context-by-cluster matrix of posterior membership probabilities from the finite-mixture model, evaluated at the reported fit. Rows correspond to contexts and columns to clusters, with each row summing to one. This matrix provides the complete posterior membership distribution, whereas `context_assignments` reports only the assigned cluster and its probability.

`mixing_proportions` A named numeric vector containing the estimated mixture proportion for each cluster. The values are nonnegative and sum to one. They represent the estimated prevalence of the latent clusters in the collection of contexts.

`gic` The generalized information criterion for the reported SMESH fit. When `G` is supplied by the user, this is the GIC for that fitted cluster resolution. When `G = NULL`, it is the GIC associated with the selected number of clusters.

`diagnostics` A named list of computational diagnostics for model fitting and cluster-number selection. The restart entries refer to the selected number of clusters and support size:

`restart_gic` The generalized information criterion (GIC) reached by each random start at the end of its EM run.

`restart_convergence` A logical vector indicating, for each random start, whether the mixture fit met the convergence tolerance within `NMAX` iterations.

`selected_restart` The index of the random start retained for the reported fit: the converged start with the smallest GIC. If no start converged, the start with the smallest GIC is retained, with a warning.

`gic_path` When `G` is selected automatically, a data frame containing the candidate values of `G` that were fitted, their GIC values, and an indicator of the selected model. `NULL` when `G` is supplied.

These quantities assess computational behavior across initializations of the same data and should not be interpreted as robustness to changes in the observed data.

Author(s)

Zhoujingpeng Wei, Guanhua Chen, Zheng-Zheng Tang

See Also

`smesh.summary`, `signature_table`, `plot_signatures`, `melody`

Examples

```
library("miMeta")
data("sim_data")
## 10 simulated contexts in 2 latent clusters, 100 features, 30 true signatures
feature.counts <- sim_data$sim_abd
sim_meta <- sim_data$sim_meta

## Disease status of each sample, one single-column matrix per context
covariate.interest <- lapply(names(feature.counts), function(d){
  m <- sim_meta[sim_meta$Context == d, ]
  matrix(m$Group, ncol = 1, dimnames = list(m$Sample_ID, "disease"))
})
names(covariate.interest) <- names(feature.counts)

########## Structure-learning meta-analysis ##########
## Each context is one study, so `context` is left at NULL. G = 2, the true
## number of clusters, keeps the example quick; with the default G = NULL,
## G is selected by the minimum GIC.
fit <- smesh(feature.counts = feature.counts,
             covariate.interest = covariate.interest, G = 2)
fit$disease$context_assignments
head(fit$disease$effects)

## The truth the data were simulated from
sim_data$sim_truth$cluster
table(sim_data$sim_truth$signatures$sharing)
```

`smesh.summary` *Structure-learning meta-analysis of context-level summary statistics*

Description

Fits the SMESH structure-learning model to user-supplied context-level summary statistics.

Usage

```
smesh.summary(
  summary.stats,
  G = NULL,
  support.ic = c("BIC", "HBIC", "KBIC", "EBIC"),
  tol = 0.001,
  NMAX = 20,
  nstart = 20,
  verbose = FALSE
)
```

Arguments

`summary.stats`	A named list of context-level summary statistics with one element per context; the list names are the context identifiers. Each element is a list with three components: `est`, a feature-by-covariate matrix of association coefficient estimates; `stderr`, the matching matrix of their standard errors; and `n`, the number of samples contributing to the context. Row names of `est` and `stderr` are microbial feature identifiers and column names are the covariates of interest. Feature identifiers must be harmonized so that the same identifier denotes the same microbial feature in every context, although a feature need not be estimable in every context: a feature that a context cannot estimate is recorded as `NA`.
`G`, `support.ic`, `tol`, `NMAX`, `nstart`, `verbose`	See function `smesh`.

Details

SMESH first learns latent context clusters jointly with sparse association effects. Conditional on the learned clustering, it estimates the final signature-sharing patterns and refits the selected cluster-level effects without penalization. A signature is all-cluster shared when its association is selected as nonzero in every cluster in which it is estimable; the estimated effects need not be equal in magnitude across clusters.

With one or two clusters, the Stage 1 basis already represents every nonempty subset of clusters. With $G > 2$, Stage 2 holds the learned clustering fixed and considers all $2^G - 1$ nonempty cluster subsets. Because the number of candidate sharing patterns grows exponentially with G, the function estimates the required memory before constructing the Stage 2 design matrix. By default, it stops if the estimated working memory exceeds 8 GB. The limit can be changed, for example, with `options(miMeta.max.stage2.gb = 16)`.

Conditional on each candidate G, the sparse support size is selected using the information criterion specified by `support.ic`. When `G = NULL`, the candidate G with the smallest Stage 1 GIC is retained. SMESH uses multiple random starts and reports the converged fit with the smallest GIC. The restart information stored in `diagnostics` describes computational behavior across initializations.

Value

Same output as the function smesh. In particular, a feature is all-cluster shared when its association is selected as nonzero in every cluster in which it is estimable; its effects need not be equal and may differ in magnitude and direction across clusters.

Author(s)

Zhoujingpeng Wei, Guanhua Chen, Zheng-Zheng Tang

See Also

smesh, signature_table, plot_signatures, melody.meta.summary

Examples

```
library("miMeta")
data("sim_data")
## 10 simulated contexts in 2 latent clusters, 100 features, 30 true signatures
feature.counts <- sim_data$sim_abd
sim_meta <- sim_data$sim_meta

## Disease status of each sample, one single-column matrix per context
covariate.interest <- lapply(names(feature.counts), function(d){
  m <- sim_meta[sim_meta$Context == d, ]
  matrix(m$Group, ncol = 1, dimnames = list(m$Sample_ID, "disease"))
})
names(covariate.interest) <- names(feature.counts)

########## Generate context-level summary statistics ##########
## One summary per context, from PALM. Alternative summary statistics can be
## used instead, as long as they give an estimate and a standard error per
## feature. A context covered by several studies would first combine them by
## fixed-effect meta-analysis, for example with PALM::palm.meta.summary().
null.obj <- PALM::palm.null.model(feature.counts = feature.counts)
summary.stats <- PALM::palm.get.summary(null.obj = null.obj,
                                        covariate.interest = covariate.interest,
                                        correct = "tune")

########## Structure-learning meta-analysis ##########
## G = 2 keeps the example quick; with the default G = NULL, G is selected by
## the minimum GIC.
fit <- smesh.summary(summary.stats = summary.stats, G = 2)
fit$disease$context_assignments
head(fit$disease$effects)
```